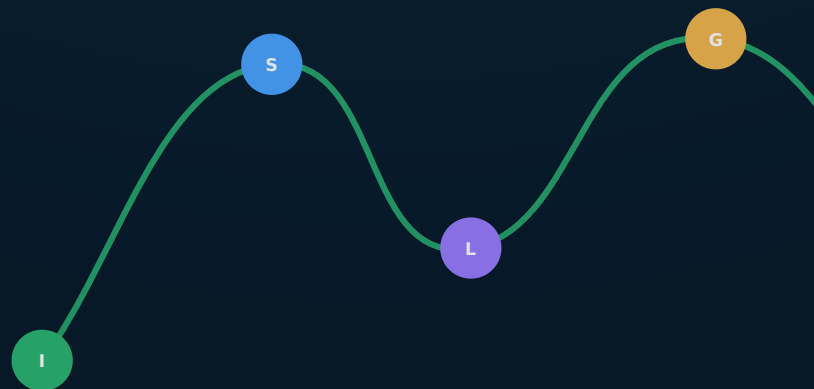


SEAMWISE DECOMPOSITION

The complete implementation blueprint

An architecture-aware, model-agnostic intent-to-task compiler for greenfield, brownfield, and hybrid engineering - with four skills, a portable CLI, embedded Task Pack, lineage graph, OTel, HTML evidence, and a governed lessons flywheel.



Intent -> Seams -> Lanes -> Legs -> Trustworthy Tasks

READ THIS FIRST

Purpose, status, and source boundary

Find the joints. Form the lanes. Stage the capability. Prove the work. Learn from every run.

Status. This is a target implementation blueprint. It preserves current source truth from Converge 0.1 and Task-Spec format 3, while specifying a proposed standalone Seamwise package and CLI.

Core decision

Task-Spec remains the atomic contract and lightweight user mode, but its implementation becomes the embedded **Task Pack** inside the Seamwise repository.

Seamwise exposes four native transformations:

[to-seam-map](#)[to-delivery-plan](#)[to-task-graph](#)[to-task-specs](#)

Source basis. Current Converge Pass 3 already defines the semantic chain seam -> swimlane -> leg -> Task-Spec. Current Task-Spec already supports runnable leaves and XL/XXL composition nodes. This document turns those semantics into an independently usable implementation architecture.

EXECUTIVE DECISION

The missing middle becomes a product

Problem

Agentic systems jump from a fuzzy feature to a flat task list. The list may be small, but it loses architecture, causal ordering, proof boundaries, and safe parallelism.

Method

Discover the real system joints. Give each joint one coherent swimlane. Sequence meaningful capability legs. Compile each leg into independently verifiable Task-Specs.

Output

A reviewed, lineage-complete Task-Spec DAG that any conformant executor or Converge runtime can consume.

Seamwise is not task splitting. It is meaning-preserving work lowering.

Canonical definition. Seamwise Decomposition is a model-agnostic engineering method and toolchain that transforms Delivery Intent plus system evidence into an architecture-aware, dependency-safe, eval-backed Task-Spec graph.

01

SECTION 01

Why Seamwise exists

The failure is not that plans are too large. The failure is that decomposition usually destroys meaning.

PROBLEM

Why ordinary decomposition is not enough

The flat-backlog failure

Feature: organization-level rate limits

- add schema
- add endpoint
- add tests
- update docs
- add metrics

The tasks may be individually readable but collectively incoherent: no architecture, no lane ownership, no capability order, no integration path, no proof lineage.

The Seamwise correction

Delivery Intent

- > policy seam
 - > policy lane
 - > configuration contract leg
 - > policy resolution leg
- > enforcement seam
 - > enforcement lane
 - > synchronous enforcement leg
 - > degraded behavior leg
- > each leg -> Task-Spec leaves

The same work becomes smaller without losing the system model that made the tasks legitimate.

Anti-pattern. Do not decompose according to how easily work can be assigned. Decompose according to how the system can safely change and how the outcome can be proven.

PRINCIPLES

Six design commitments

Architecture first

- Cut where the system is already jointed.
- A seam must have a coherent responsibility and interface.
- Team names and folder names are not sufficient evidence.

Intermediate value

- Every leg leaves a meaningful capability state.
- Activities such as “design” and “test” are not capability legs.
- A steel thread proves early end-to-end value.

Atomic proof

- Every executable leaf has one coherent done-condition.
- Proof travels with the atom.
- Parallelism is earned through dependencies and write isolation.

Model agnostic

- The compiler outputs contracts, not vendor prompts.
- Any conformant engine can implement the leaf.
- Adapters and backend metadata stay at the edge.

Fail closed

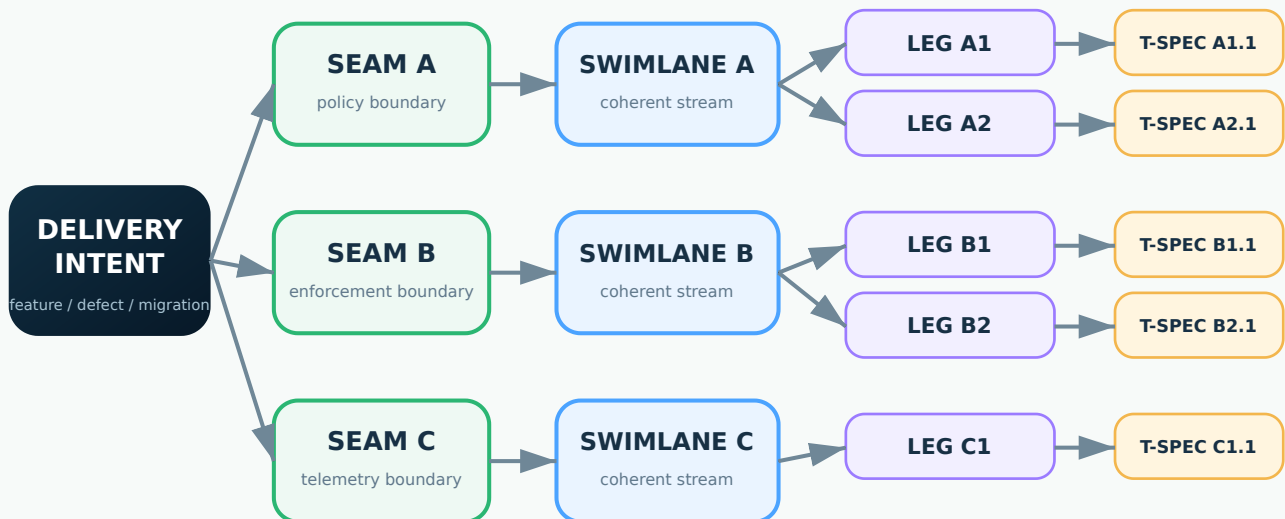
- Ambiguous discovery does not become optimistic planning.
- Unresolved owner or architecture decisions stop at a named gate.
- No green verdict is forced when the evidence is missing.

Continuous learning

- Every transformation emits lineage and telemetry.
- Lessons are reviewed, scoped, versioned, benchmarked, and reversible.
- No lesson silently weakens a signed contract.

VOCABULARY

The five durable concepts



Structure -> coordination -> progression -> proof

LEVEL	CANONICAL MEANING	REQUIRED PROPERTY	FAILURE SMELL
Delivery Intent	The outcome to make true	Actor/system, trigger, observable outcome, scope, constraints, evidence	A solution request with no measurable result
Seam	A natural system joint	Evidence, responsibility, interface, independence claim	A folder/team split or a vague "area"
Swimlane	One coherent stream aligned to a seam	Owner, inputs/outputs, non-responsibility, dependencies	A lane that needs another lane's internals
Leg	An observable capability increment	Ordered state change, proof seed, inherited decisions	Activity labels such as design/test/cleanup
Task-Spec	One independently provable atom or composition node	Scope, behavior, evals, budgets, lifecycle	A ticket whose done condition lives in a reviewer's head

INVARIANTS

The type system that keeps the graph honest

Canonical cardinality

```
1 Delivery Intent
  -> 1..N accepted seams

1 accepted seam
  -> exactly 1 owning swimlane

1 swimlane
  -> 1..N ordered capability legs

1 leg
  -> 1..N task candidates

1 task candidate
  -> exactly 1 Task-Spec node or leaf
```

Strict type discipline

- Avoid nested legs in version one. If a leg needs sub-legs, it is likely oversized or hiding another seam.
- Cross-cutting concerns become constraints, interfaces, or explicit integration tasks - not duplicate ownership.
- A leaf owns one coherent failure story.
- Sibling tasks are not automatically parallel.

Governing law. Every Task-Spec inherits its why from the leg, its coordination from the swimlane, its boundary from the seam, and its proof from the Task Pack.

TERRAIN

Greenfield, brownfield, and hybrid

Greenfield

There is little or no implementation to inherit. Discovery grounds the intent in surrounding systems, protocols, data sources, deployment constraints, and explicit assumptions. New files must be represented as creates paths.

Brownfield

The live repository is authoritative evidence. Discovery opens schemas, services, jobs, tests, deployment and operational paths end to end. Seams must reflect existing contracts, not wishful diagrams.

Hybrid

A new capability lands inside an existing environment. Some seams inherit real contracts; others are deliberately created. The seam map distinguishes observed joints from new architectural decisions.

TERRAIN	STRONG EVIDENCE	TYPICAL STOP
Greenfield	Source-system contracts, non-functional constraints, deployment target, explicit assumptions	Missing owner outcome or hard-to-reverse architecture choice
Brownfield	Repository paths, schemas, tests, runtime flows, ownership and operational evidence	Live system contradicts the brief or no defensible seam exists
Hybrid	Both live-system evidence and explicit new-boundary decisions	A proposed seam requires an unrecorded architecture decision

02

SECTION 02

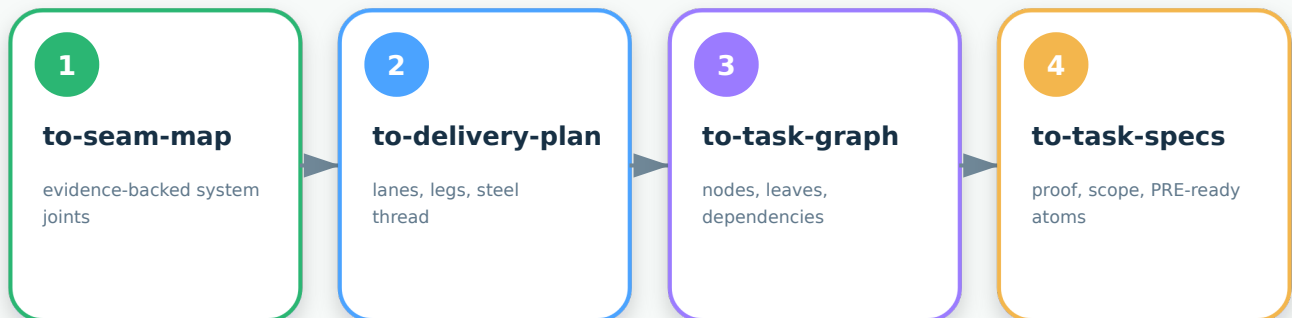
System architecture

Four skills, one Task Pack, one CLI, and a clean handoff to any execution system.

ARCHITECTURE OVERVIEW

The final 4-skill design

Four transformations. One intent-to-task compiler.



The first three decide what the work is. The fourth materializes each trustworthy atom.

User-facing orchestrator

/seamwise or seamwise prepare
inspects the artifact state, invokes only the missing transformations, stops at gates, and never silently crosses owner decisions or PRE stamping.

Reusable skills

Each transformation can be called directly by an agent or harness when its input artifact already exists. Skills are artifact transformers, not persistent personas.

PACKAGING

Task-Spec moves inside without losing its identity

Seamwise Core

```
skills/  
  seamwise/          #  
orchestrator  
  to-seam-map/  
  to-delivery-plan/  
  to-task-graph/  
  to-task-specs/  
  
task-pack/  
  format/  
  templates/  
  validators/  
  gates/  
  lifecycle/  
  conformance/
```

Two CLI profiles, one implementation

```
bin/seamwise  
  complete intent-to-task compiler  
  
bin/task-spec  
  thin atomic-mode wrapper over  
  seamwise tasks ...
```

Task-Spec stays the artifact and brand. Task Pack is the embedded implementation subsystem.

Source-of-truth rule. There is one implementation of the format parser, template, validator, PRE, POST, HMAC, lifecycle, graph extraction, and conformance tests.

03

SECTION 03

Skill 1 - to-seam-map

Transform Delivery Intent plus evidence into defensible, machine-readable system joints.

TO-SEAM-MAP

Contract and inputs

Contract

INPUT

Delivery Intent
repository / system evidence
optional accepted requirements and
decisions

OUTPUT

system map
evidence register
accepted/rejected seam records
open decision register

Accepted starting inputs

- Idea, pain, request, feature, defect, migration, architecture initiative.
- BRD, technical requirements, accepted Converge spec and ADRs.
- Greenfield constraints and surrounding-system contracts.
- Existing repository and operational evidence.

Purpose. Understand enough of the outcome and the real terrain to identify genuine system joints without falling to task or code altitude.

TO-SEAM-MAP

Process I - intent and discovery

1. Normalize Delivery Intent

- Affected actor or system.
- Trigger and observable outcome.
- Scope and non-goals.
- Constraints and acceptance evidence.
- Unresolved decisions with owners.

2. Classify terrain

- greenfield
- brownfield
- hybrid

3. Discover reality

- Repository structure, schemas, services, jobs.
- Interfaces, protocols, data and ownership boundaries.
- Deployment, security and operational constraints.
- Authoritative docs and relevant external sources.

4. Record evidence

- Source URI/path and content hash.
- Observation, freshness and confidence.
- Fact, assumption, or owner decision.
- Which seam hypothesis the evidence supports or refutes.

TO-SEAM-MAP

Process II - hypothesis and refutation

5. Generate seam hypotheses

- Feature boundary.
- Component or protocol boundary.
- Data ownership or trust boundary.
- Deployment or operational responsibility.
- Separately reversible or provable outcome.

6. Attack each hypothesis

- Do both sides have distinct responsibilities?
- Can the boundary be described through an interface?
- Can each side be reasoned about or proven independently?
- Is this merely a team, directory, or arbitrary slice?
- Does one side secretly require the other side's internals?

Repair rule. A fuzzy seam is repaired by moving or folding the boundary, or by returning to the architecture decision that owns it. Adding prose around a false boundary does not make it real.

TO-SEAM-MAP

Artifact model and seam schema

Authoritative seam record

```
---
id: seam-policy
status: accepted
terrain: hybrid
owner: platform-policy
confidence: 0.93
consumes: [tenant-context]
produces: [effective-policy]
evidence:
  - repo://src/policy/schema.py
  - adr://ADR-004
rejected_alternatives:
  - team-based split
---
# Responsibility
Resolve effective organization policy.

# Independence claim
Can be proven without request
enforcement.
```

Artifact family

```
seamwise/
  intent.md
  system-map.md
  evidence.jsonl
  decisions/
    seams/S-*.md
    seam-map.yaml      # rebuildable
  projection
  open-decisions.yaml
```

Authored Markdown with typed frontmatter is canonical. Aggregate YAML is a rebuildable view unless the implementation explicitly chooses otherwise.

TO-SEAM-MAP

Gate and failure routes

TOKEN	MEANING	CORRECT RECOVERY
SEAM_MAP=READY	Every accepted seam is evidence-backed and meaningful	Proceed to delivery planning
SEAM_MAP=NEEDS_DISCOVERY	The answer is discoverable but evidence is insufficient	Research the named sources and rerun
SEAM_MAP=NEEDS_OWNER_INPUT	A product or scope decision is missing	Ask only the named owner decision
SEAM_MAP=NEEDS_ARCHITECTURE_DECISION	A hard-to-reverse choice is missing	Record/approve the decision, then renew
SEAM_MAP=AMBIGUOUS	The available evidence cannot establish a truthful boundary	Stop; do not manufacture a seam
SEAM_MAP=ERROR	The mechanism or artifact shape failed	Repair tooling or malformed input

Gate definition of done. Every accepted seam has supporting evidence, one coherent responsibility, consumed and produced interfaces, ownership, an independence claim, and recorded rejected alternatives.

04

SECTION 04

Skill 2 - to-delivery-plan

Turn accepted seams into coherent swimlanes, ordered capability legs, a steel thread, and a reviewed delivery graph.

TO-DELIVERY-PLAN

Contract and responsibilities

Contract

INPUT

accepted seam records
Delivery Intent
evidence and decisions
shared vocabulary

OUTPUT

swimlanes
capability legs
steel thread
dependency + contention graph
adversarial review receipt

Core purpose

- Assign exactly one owning swimlane per seam.
- Define interfaces, responsibilities and non-responsibilities.
- Sequence capability states rather than activities.
- Reveal real dependencies and safe parallelism.
- Harden the plan before task compilation.

TO-DELIVERY-PLAN

Swimlanes - coordination aligned to structure

Swimlane shape

```
id: lane-policy
seam: seam-policy
responsibility: resolve effective
policy
non_responsibilities:
  - request enforcement
consumes:
  - tenant-context
produces:
  - effective-policy
owner: platform-policy
depends_on: []
```

Lane test

- One coherent responsibility.
- A clear interface and owner.
- The lane can progress without inspecting another lane's internals.
- Dependencies are explicit edges, not narrative.
- Completion is meaningful beyond "tasks are done."

One lane per accepted seam. A seam names the structural joint. The swimlane names the coherent stream of delivery that follows it.

TO-DELIVERY-PLAN

Capability legs - progression inside a lane

Capability leg shape

```
id: leg-policy-resolution-01
lane: lane-policy
title: Effective policy resolves with
explicit precedence
inherits:
  requirements: [FR-2]
  decisions: [ADR-004]
consumes: [policy-records]
produces: [effective-policy]
proof_seed:
  given: org and default policy exist
  when: resolution is requested
  then: documented precedence is
  returned
independently_finishable: true
```

Leg test

- Names a capability state, not an activity.
- Creates observable intermediate value.
- Has inherited decisions and a proving seed.
- Fits inside one lane.
- May compile into one or many Task-Specs.

Bad legs. “Design”, “implementation”, “testing”, and “cleanup” describe work being performed. They do not describe a capability that now exists.

TO-DELIVERY-PLAN

Steel thread, dependencies, and parallel safety

The steel thread

The minimum set of legs across the minimum set of lanes that proves end-to-end value early.

- Not a demo-only shortcut.
- Uses production-relevant interfaces and proof.
- Surfaces integration risk before every lane is “complete.”
- Becomes an explicit critical-path and integration-test target.

Example

Policy schema

- > Policy resolution
- > Request enforcement
- > Decision telemetry

Proof:

Given an org with a 100 rpm policy,
When request 101 arrives,
Then it is rejected and a visible
reason code is emitted.

Parallelism is earned. Lanes can be explored in parallel. Write-heavy execution is parallel only when dependency and path-contention analysis says it is safe.

TO-DELIVERY-PLAN

Adversarial decomposition review

Adversarial reviewer attacks

- False seams and hidden coupling.
- Missing interfaces or architectural decisions.
- Legs that are not independently finishable.
- Proof seeds that can be Goodharted.
- Fake parallelism and shared-file contention.
- Failure modes normalized away by the author.

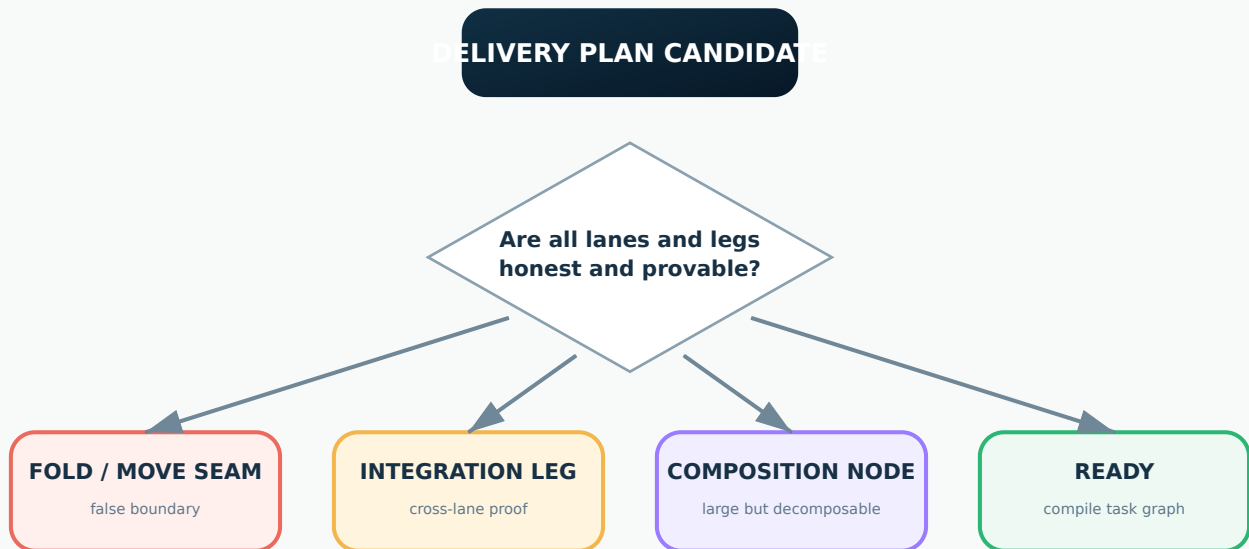
Every objection ends as

- FIXED - the plan changed and points to the repair.
- ACCEPTED - the risk remains intentionally with a named owner and rationale.
- OPEN - the gate remains closed.

Review is not the gate. The different-family attack creates useful disagreement. Closure requires every objection resolved against the current plan bytes and, when embedded in Converge, the owner barrier.

TO-DELIVERY-PLAN

Decomposition Resolution Gate



No dependency on Converge. Seamwise owns decomposition resolution, not ceremony routing. Missing owner input or architecture decisions become named stop states. Embedding systems decide how to satisfy those stops.

05

SECTION 05

Skill 3 - to-task-graph

Compile reviewed capability legs into a semantic graph of composition nodes, executable leaves, dependencies, lineage, and safe parallelism.

TO-TASK-GRAPH

Contract and responsibilities

Contract

INPUT

reviewed delivery plan revision
review receipt and hashes
target repository / workspace

OUTPUT

semantic task candidates
parent-child graph
dependency edges
critical path and ready frontier
path-contention analysis
complete lineage

It decides

- Which work units must exist.
- Which are executable leaves vs composition nodes.
- Which edges express prerequisites.
- Where cross-lane integration proof is required.
- Which tasks may safely run concurrently.

TO-TASK-GRAPH

Nodes, leaves, edges, and lineage

Task graph candidate

```
task_graph:
  root:
    id: node-rate-limiting
    kind: composition-node
    children:
      - policy-schema
      - policy-resolver
      - enforcement
      - telemetry

    tasks:
      - id: policy-schema
        kind: leaf
        leg: leg-policy-contract
        depends_on: []
        outcome: invalid policy is
        rejected
        scope_hints:
          - src/policy/schema.py
          - tests/policy/test_schema.py
```

Required lineage

- Delivery Intent and owner outcome.
- Requirement IDs and architecture decisions.
- Seam, swimlane and leg.
- Review receipt and source hashes.
- Proposed Task-Spec ID and format version.

Semantic graph first. Skill 3 defines which atoms should exist. Skill 4 defines what each trustworthy atom contains.

TO-TASK-GRAPH

Graph semantics and atomicity

GRAPH ELEMENT	RULE	WHY
Runnable leaf	One coherent done-condition, independently provable, XS-L	Can be delegated and accepted on its own
Composition node	XL/XXL, no write surface, never dispatched	Preserves large intent without pretending it is atomic
depends_on edge	Represents real prerequisite or intentional serialization	Makes readiness machine-readable
parent-child edge	Preserves composition and lineage	Lets accepted leaves satisfy a larger outcome
integration leaf	Owens cross-lane proof or final contract	Makes coupled value explicit rather than hidden
contention edge	Overlapping write surface or exclusive resource	Prevents unsafe “parallel because siblings” behavior

Leaf test. If one eval can fail while the remaining work is still independently shippable, the candidate contains multiple atoms.

TO-TASK-GRAPH

Critical path, readiness, and contention

Critical path

Compute the longest prerequisite chain using expected effort/duration hints, but treat it as a planning projection rather than canonical truth.

```
schema -> resolver -> middleware -> steel-  
thread proof
```

Ready frontier

Candidate leaves whose dependencies and declared preconditions are satisfied. Dispatch policy must still validate leaf effort, Task-Spec PRE/trust, blocked reason, and runtime capability.

Path contention rules

- Exact path overlap requires sequencing or redesign.
- Shared generated files and schema registries may be exclusive resources even when paths differ.
- Globs must not hide the real surface.
- A read-only exploration worker may run in parallel with a writer only if the environment and tools preserve isolation.
- No graph claims parallel safety without evidence.

TO-TASK-GRAPH

Gate and failure routes

TOKEN	MEANING	RECOVERY
TASK_GRAPH=READY	Graph is acyclic, lineage-complete, and each candidate is materializable	Proceed to Task-Spec materialization
TASK_GRAPH=CYCLE	Prerequisite graph is cyclic	Redesign dependencies; never mark a predecessor done by hand
TASK_GRAPH=COLLISION	Write/resource contention invalidates proposed parallelism	Sequence or change the slice
TASK_GRAPH=UNPROVABLE_NODE	A proposed leaf lacks an honest independent proof boundary	Split, add integration leg, or make it a composition node
TASK_GRAPH=BLOCKED	Open decision/precondition prevents truthful compilation	Resolve the named stop
TASK_GRAPH=ERROR	Schema, parser or mechanism failed	Repair the implementation

Definition of done. Every task candidate has stable identity, kind, lineage, dependencies, scope hints, expected outcome, and a clear materialization route.

06

SECTION 06

Skill 4 - to-task-specs

Materialize every semantic node and leaf into the correct Task-Spec shape, complete with proof, authority, lifecycle, and explicit PRE modes.

TO-TASK-SPECS

Contract and the 3+1 split

Contract

INPUT

valid semantic task graph
target repository
embedded Task Pack

OUTPUT

canonical Task-Spec nodes and leaves
behavior and eval traceability
scope, dependencies and budgets
Task-Spec DAG projections
validation / preflight evidence

Why this is a separate skill

- Skill 3 decides which atoms must exist.
- Skill 4 chooses the correct Task-Spec profile and effort shape.
- It authors behavior, evals, Validation Card and Exit Check.
- It binds authority, lifecycle and trust fields.
- It runs validation and optional PRE sealing.

TO-TASK-SPECS

Task-Spec as the embedded atomic contract

YAML frontmatter

identity, graph, scope, budgets, trust

1. Intent

why + goal + lean context

2. Behavior

stable B-N observable promises

3. Contract

evals + validation card + exit check

4. Guardrails

anti-patterns + do-not-touch

5. Operations

open questions + blockers

6. Reversal/runtime

rollback + observability

The center of gravity is executable proof, not prose.**Intent**

One bounded goal, lean context, lineage and explicit non-goals.

Behavior + proof

Stable B-N promises, evals, Validation Card, retry policy, agent contract and Exit Check.

Authority + operations

Paths, dependencies, budgets, do-not-touch, open questions, rollback and observability.

TO-TASK-SPECS

Choosing the correct shape

AXIS	CHOICES	DECISION
Effort	XS, S, M, L	Runnable leaf with one coherent done-condition
Effort	XL, XXL	Non-runnable composition node with children and zero write surface
Profile	lite	Small, low-blast work whose evals express behavior directly
Profile	standard	Default; context, behavior, guardrails, operations and complete traceability
Profile	full	Unattended, hard-to-reverse, security, financial or operationally material work

Escalate rigor when

- Unattended or overnight execution.
- Difficult rollback or destructive effects.
- Security or money.
- Runtime alerts and recovery must be explicit.

Do not over-escalate

- Atomicity is set by effort, not profile.
- A full parent does not force every child to full.
- Choose the lightest profile that still tells the truth.

TO-TASK-SPECS

Materialization modes and mutation boundaries

draft

Create canonical Task-Spec files and graph projections. No readiness claim.

validate

Run format, sizing, traceability, dependency and graph checks. No autonomy envelope.

preflight

Run PRE mechanics and verdict mode without stamping. Evals may still have side effects.

seal

Explicit mutation that writes the PRE envelope and HMAC when available.

```
seamwise tasks emit
seamwise tasks validate
seamwise tasks preflight
seamwise tasks seal

# lightweight compatibility profile
task-spec new ...
task-spec validate ...
task-spec gate --stamp ...
```

Never auto-seal as a side effect of ordinary compilation. Only an explicit gate operation turns reviewed contracts into dispatch authority.

07

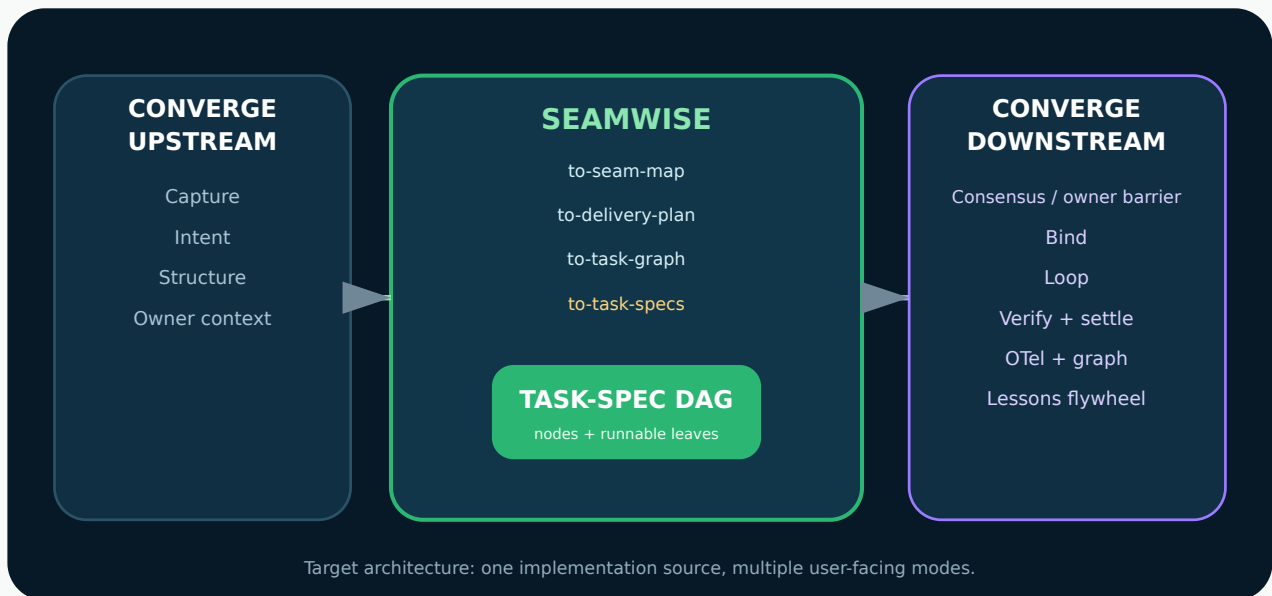
SECTION 07

End-to-end system

CLI, workspace, artifact authority, observability, graph, reports, and lessons.

END-TO-END

Standalone Seamwise and embedded Converge modes



Standalone flow. Delivery Intent + system evidence -> four Seamwise skills -> ready Task-Spec DAG -> any conformant executor.

Embedded flow. Converge provides accepted intent/structure, invokes Seamwise, then adds Consensus/owner barrier, Bind, Loop, settlement, graph and learning.

CLI

Agent-native, parse-stable, provider-neutral

Human command surface

```
seamwise init
seamwise map
seamwise plan
seamwise review
seamwise compile
seamwise prepare
seamwise tasks
emit|validate|preflight|seal
seamwise inspect
seamwise graph
seamwise report
seamwise agent-context
```

Command contract

- Explicit workspace and artifact paths.
- Stable token plus exit code.
- JSON envelope for automation.
- Dry-run semantics declared per mutation.
- Package and protocol versions.
- Self-description through agent-context.
- Provider injection for models, research, storage, Task Pack, and telemetry.

CLI rule. Wrap behavior-bearing scripts and libraries; do not reimplement their semantics in the router.

WORKSPACE

One decision, one durable home

Canonical workspace

```
seamwise/  
  intent.md  
  system-map.md  
  evidence.jsonl  
  decisions/  
  seams/  
  seam-map.yaml  
  swimlanes/  
  legs/  
  steel-thread.md  
  reviews/  
  
tasks/  
  T-*.md  
  task-graph.yaml  
  task-lineage.json  
  critical-path.mmd  
  preflight.json  
  
telemetry/  
reports/  
lessons/
```

Authority classes

CLASS	EXAMPLES	RULE
Canonical authored truth	intent, decisions, seam records, lane/leg records, Task-Specs	Changed only by the owning transformation and renewed review
Derived local state	aggregate YAML, indexes, critical path, graph projections	Rebuildable from canonical sources
Append-only evidence	telemetry events, lifecycle and attempt logs	Corrected by later accounted events
External projection	tracker, PR, cockpit, HTML report	Reconciled from canonical identity and evidence

OBSERVABILITY

OTel semantic model

```
seamwise.run
  seamwise.to_seam_map
    discovery.repository
    discovery.documents
    seam.propose
    seam.gate
  seamwise.to_delivery_plan
    lane.form
    leg.sequence
    steel_thread.select
    objection.raise
    delivery_plan.gate
  seamwise.to_task_graph
    task.classify
    task.edge
    collision.check
    lineage.gate
  seamwise.to_task_specs
    taskspec.compose
    taskspec.validate
    taskspec.pre
  seamwise.report.build
  seamwise.lesson.extract
```

Trace attributes

run.id, repo.sha,
skill.name/version,
artifact hashes,
model/provider, seam.id,
lane.id, leg.id, task.id.

Events

objection
raised/resolved, retry,
eval verdict, path
collision, owner stop,
lesson candidate,
promotion.

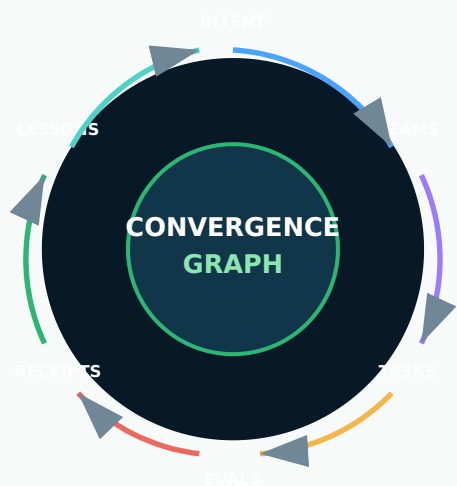
Metrics

cycle time, discovery
budget, seam churn, leg
rework, proof coverage,
retry cost, collisions,
acceptance rate.

Artifacts authorize. OTel observes. Telemetry must never turn a failure into success or overwrite canonical truth.

CONVERGENCE GRAPH

Lineage from owner outcome to accepted evidence



OTel records every transformation and every gate

- spans: discovery, decomposition, task compilation, Bind, Loop, verification and POST
- attributes: artifact hashes, model, skill version, seam, lane, leg and task identity
- events: objections, retries, verdicts, path checks and lesson candidates

Artifacts authorize. OTel observes.

The graph connects. The flywheel improves.

Promoted lessons are scoped, reviewed, versioned, tested, measured, reversible, and never self-approving.

NODE	EXAMPLES	KEY EDGES
Intent / Requirement / Decision	owner outcome, FR-3, ADR-004	DERIVES_FROM, CONSTRAINED_BY
Seam / Lane / Leg	policy seam, enforcement lane, degraded-mode leg	SUPPORTED_BY, CONTAINS, CONSUMES, PRODUCES
Task / Run / Eval	Task-Spec leaf, attempt, holdout verdict	DEPENDS_ON, ATTEMPTED_BY, EVALUATED_BY
Receipt / Acceptance	settlement receipt, POST stamp	RECEIPTED_BY, ACCEPTED_AS
Lesson / SkillVersion	retry-idempotency lesson, to-seam-map@0.1	LEARNED_FROM, PROMOTED_TO

REPORTS AND COCKPIT

Human-readable lineage over machine-readable truth

Static HTML evidence bundle

```
reports/<delivery-id>/  
  index.html  
  data.json  
  graph.json  
  trace-summary.json  
  artifacts/  
  receipts/
```

For any task, the report should show why it exists, which requirement/decision/seam/lane/leg produced it, which revision and runtime acted, which eval failed, which paths changed, which receipt enabled acceptance, and which lessons resulted.

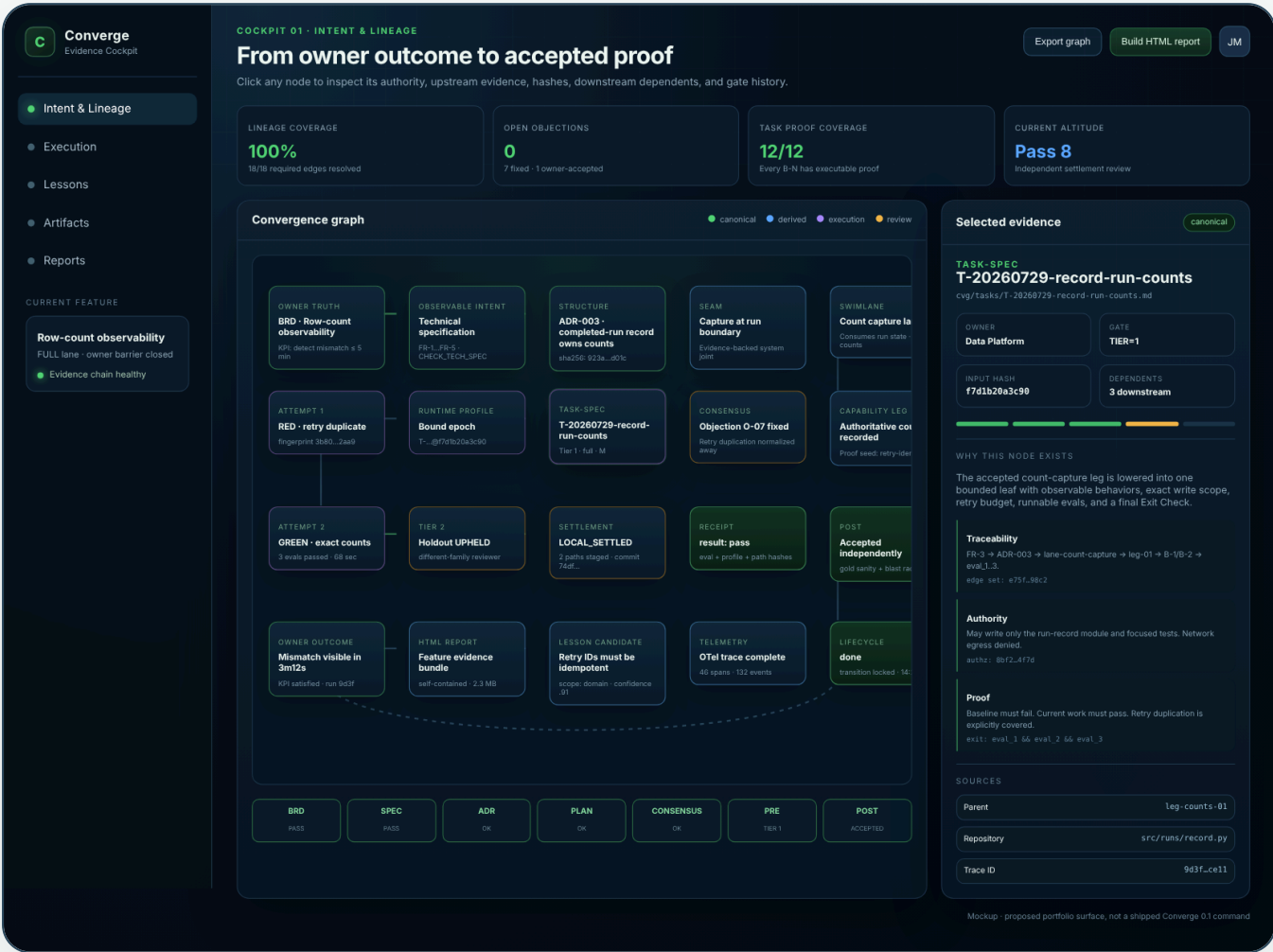
Read-only cockpit principles

- Every surface backlinks to the canonical artifact.
- Hashes and freshness are visible.
- Derived graph state can be rebuilt.
- No dashboard edits upstream truth.
- Roadmap capabilities are styled as future.

The HTML is not decoration. It is the human navigation layer over the evidence graph.

COCKPIT MOCKUP

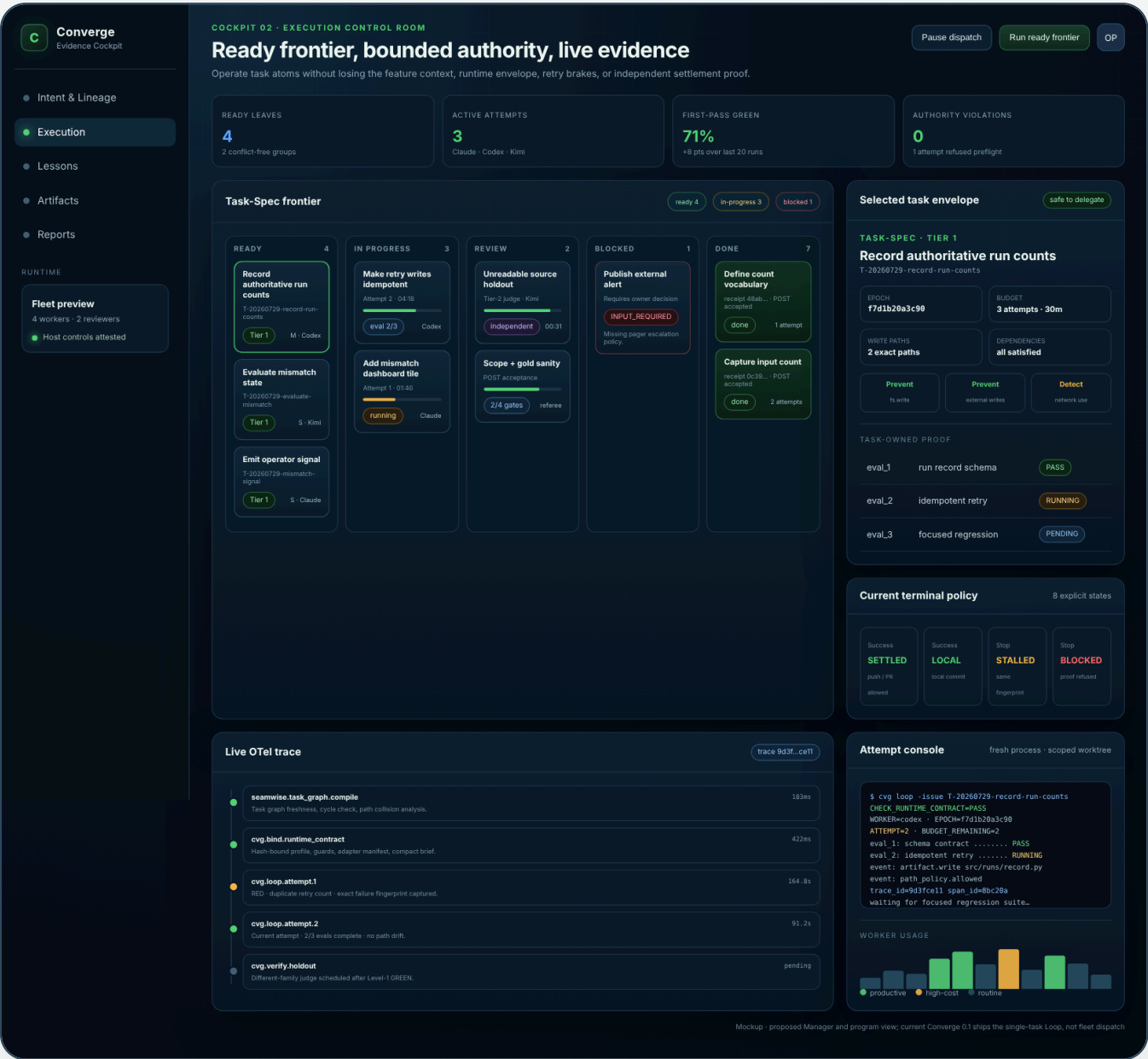
Intent and lineage



Mockup 1 - Intent and Lineage Cockpit. Click any node to inspect authority, hashes, evidence, dependents and gate history.

COCKPIT MOCKUP

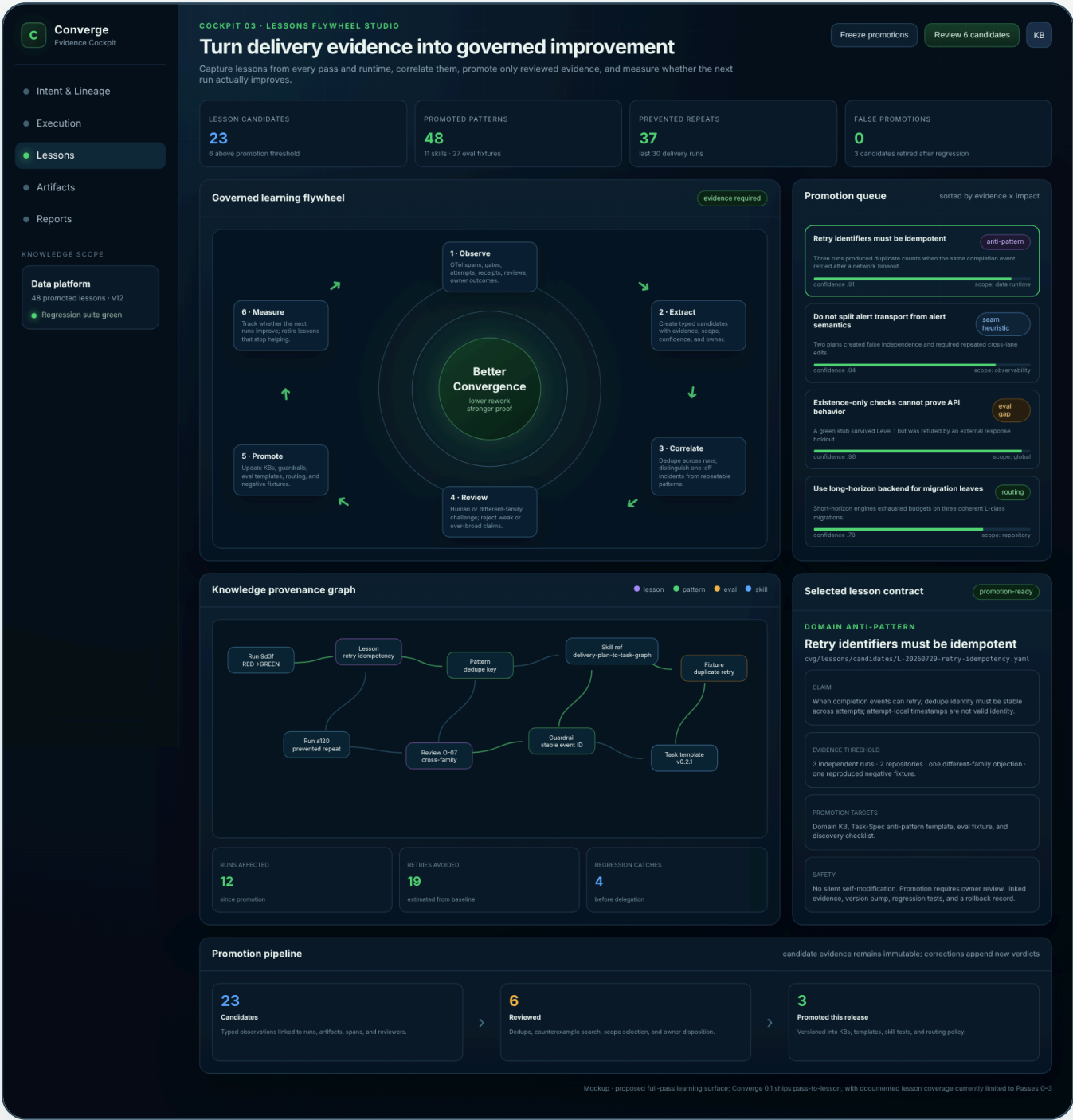
Execution control room



Mockup 2 - Execution Control Room. Ready frontier, bound authority, live attempts, evals, terminal state and OTEL spans.

COCKPIT MOCKUP

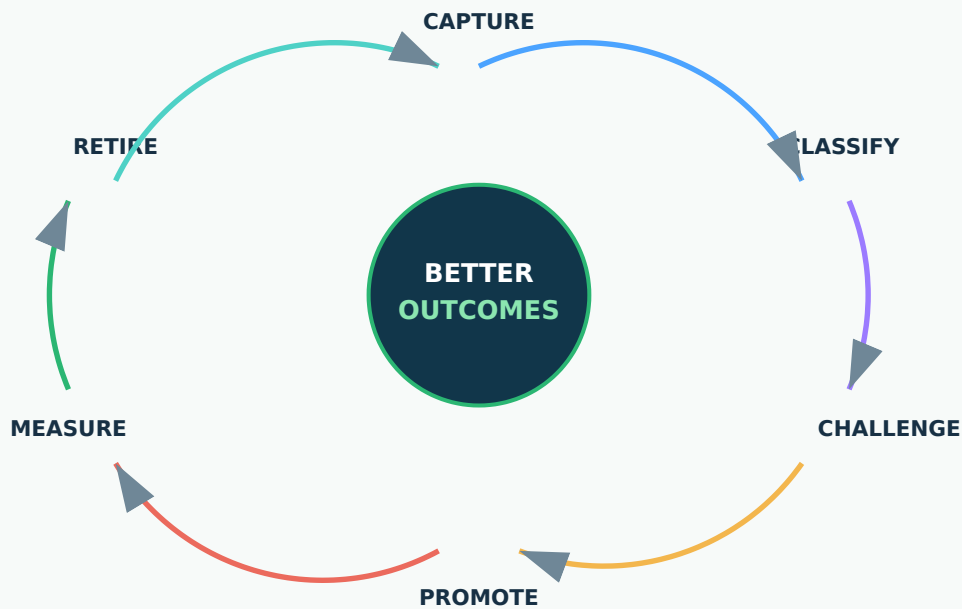
Lessons flywheel studio



Mockup 3 - Lessons Flywheel Studio. Candidates, provenance, scope, promotion targets, fixtures, measured impact and retirement.

LESSONS

The governed learning flywheel



Promotion is governed. No lesson self-approves. One failure does not become a global rule. Every promotion is scoped, linked to evidence, versioned, regression-tested, measured on later runs, and retired when stale or harmful.

LESSONS

Schema, promotion, measurement, retirement

LESSON CLASS	EXAMPLE	PROMOTION TARGET
Repository fact	Canonical request pipeline lives in src/runtime	Project context / seam discovery reference
Domain pattern	Stable retry identity must be idempotent	Pattern catalog + task template
Anti-pattern	Existence-only eval passes without behavior	Eval lint and negative fixture
Decomposition signal	Team-based seam caused cross-lane internals	Seam review checklist
Eval gap	Green visible test missed false-health path	Holdout template / gold sanity
Routing signal	Long-horizon backend performed better on L leaves	Backend recommendation policy
Operational lesson	STOP file invisible across worktree boundary	Recovery runbook and conformance test

```
lesson_id: L-20260729-retry-idempotency
source_runs: [run-9d3f, run-41aa]
class: anti-pattern
scope: domain
confidence: 0.91
claim: stable event identity must make retry writes idempotent
proposed_targets:
  - references/patterns/idempotent-writes.md
  - tests/fixtures/retry-duplicate.yaml
status: candidate
```


08

SECTION 08

Trust, packaging, and implementation

Make the system portable, testable, fail-closed, and honest about what is current versus proposed.

TRUST MODEL

Sign the contract, enforce the runtime, distrust the claim

Instruction integrity

Task-Spec PRE seals body and authority-bearing fields. A changed scope, dependency, budget or eval is a new delegation decision.

Runtime enforcement

Seamwise records requirements; only a runtime can prevent network, filesystem, secret, or tool violations. Converge Bind operationalizes this later.

Outcome evidence

Task-owned evals, optional holdout, path checks, receipts and POST are separate claims. No single green signal proves everything.

Research/MCP boundary

- Providers supply fresh evidence.
- Every source is recorded with URI/path, freshness and hash where possible.
- Perishable knowledge never silently becomes durable system truth.
- Credentials remain at adapter boundaries.

Model boundary

- A model can research, propose, attack, or author within a skill seat.
- A model is never the canonical source of truth.
- Different-family review is useful but not infallible.
- Stable tokens and exit codes govern automation.

VERSIONING

One repository, multiple independent contracts

Independent version axes

```
seamwise_release: 0.1.0
contracts:
  seam_map_version: 1
  delivery_plan_version: 1
  task_graph_version: 1
  task_spec_format_version: 3
  agent_contract_version: 2
  hmac_scheme: hmac-sha256-v2
  otel_semconv_version: 1
  convergence_graph_version: 1
```

Distribution profiles

- Task-Spec CLI - atomic mode, same embedded Task Pack.
- Seamwise CLI - complete intent-to-task compiler.
- Converge - pins Seamwise and adds owner governance, Bind, Loop, settlement, graph aggregation and learning.
- No separately edited validators or templates in packaging mirrors.

Separate release from contract version. Seam discovery can improve without invalidating Task-Spec format 3. A format migration must be explicit even if the package release number is unchanged.

TESTING

Conformance before confidence

Skill conformance

- Positive fixtures for greenfield, brownfield and hybrid.
- Adversarial fixtures for false seams, hidden coupling, unprovable legs and stale hashes.
- Schema validation and exact token/exit tests.
- Deterministic artifact generation from pinned evidence.

Task Pack conformance

- Task-Spec parser, validator, PRE, POST and lifecycle parity.
- XS-L leaf and XL/XXL node fixtures.
- Behavior-eval traceability in both directions.
- HMAC v2, tamper and Tier-2/Tier-3 negative cases.

Cross-skill e2e

- Delivery Intent -> seam map -> delivery plan -> graph -> sealed leaves.
- Upstream byte change invalidates review/lineage and requires renewal.
- Path collision prevents unsafe parallel readiness.
- Every task backlinks through leg/lane/seam to source intent.

Operational drills

- Research provider unavailable.
- Ambiguous repository discovery.
- Model timeout or malformed output.
- Eval passes baseline.
- Out-of-scope diff.
- Lesson promotion worsens benchmark and is retired.

ROADMAP

A safe implementation sequence

PHASE	IMPLEMENTATION RESULT	EXIT EVIDENCE
0 - Extract without change	Move Task Pack into Seamwise; preserve current format, scripts, tests, tokens and signatures	Byte/behavior parity with existing Task-Spec entry points
1 - Introduce four skills	Add skill contracts and artifact schemas	Each skill passes positive and adversarial fixtures
2 - Add CLI profiles	bin/seamwise and thin bin/task-spec wrapper	Command tree, stable tokens, JSON, dry-run and agent-context
3 - Add lineage + graph	Task lineage, OTel events, graph and HTML report	End-to-end trace for one proving deliverable
4 - Add flywheel	Candidate capture, review, fixtures, promotion and measurement	Promoted lesson changes behavior and is benchmarked
5 - Converge integration	Converge pins Seamwise and wraps without rewriting	FULL descent to Task-Spec DAG, Bind/Loop handoff and compatible evidence

Migration rule. Move first, refactor second. Do not rewrite Task-Spec while changing its repository ownership.

09

SECTION 09

Worked examples

See the same compiler operate at atomic, deliverable, and complete-system scales.

EXAMPLES

Health endpoint and rate limiting

Atomic mode

Intent: add a health endpoint.

```
task-spec new add-health-endpoint S
# B-1: HTTP 200 + fixed JSON
# eval_1 startup
# eval_2 exact response
# eval_3 regression
# Exit Check chains all three
```

No seam map is needed because the atom is already understood.

Seamwise direct mode

Intent: add organization-level rate limiting.

```
seamwise prepare --intent
"organization-level rate limits"

seamwise graph
seamwise tasks preflight
seamwise tasks seal
```

The compiler discovers policy, enforcement, metering and observability boundaries before materializing tasks.

WORKED EXAMPLE

Rate limiting: intent to Task-Spec DAG

Delivery Intent: organization-level API rate limiting.

Seams

Policy/config
Enforcement
Metering
Observability

Swimlanes

Policy lane
Enforcement
lane
Data lane
Telemetry lane

Legs

Schema
Resolution
Sync
enforcement
Degraded mode
Atomic recording
Decision
telemetry

Leaves

Schema
validator
Policy repository
Resolver
Middleware
Fallback policy
Usage store
Metrics/logs

Steel thread. Define schema -> resolve effective policy -> enforce request 101 -> emit a visible reason code. That path proves end-to-end value before every operational feature is complete.

Integration leaf. Cross-lane proof belongs to a named Task-Spec with its own evals, not an implicit assumption that separately green leaves compose.

WORKED EXAMPLE

Row-count observability: Seamwise inside Converge

Delivery Intent: a successful pipeline can publish incomplete output while reporting health. Make mismatch visible to an operator within five minutes.

Seamwise output

- Seam A: capture authoritative counts at the completed-run boundary.
- Seam B: evaluate mismatch and expose operator state.
- Lane A legs: attach counts; make retries idempotent.
- Lane B legs: stable mismatch state; visible signal.
- Steel thread: one completed run -> recorded counts -> mismatch -> operator signal.

Task graph

```
XL node: row-count observability
M leaf: record authoritative counts
S leaf: make retry writes idempotent
M leaf: evaluate mismatch state
S leaf: emit operator-visible signal
S leaf: prove steel thread
```

Converge handoff. When used inside Converge, the reviewed plan crosses the owner barrier, Task-Spec leaves are sealed, Bind narrows authority, Loop iterates with exact failure evidence, holdout review may refuse settlement, and POST plus a receipt enable done.

COMPARISON

Task-Spec, Seamwise, and Converge

DIMENSION	TASK-SPEC MODE	SEAMWISE	CONVERGE
Unit	One atom	One deliverable	Complete operating lifecycle
Begins	Known bounded job	Delivery Intent + system evidence	Idea, pain, BRD or high-trust need
Ends	Valid/sealed/accepted Task-Spec	Ready Task-Spec DAG	Settled, inspectable evidence chain
Core uncertainty	What exactly is done?	What are the right atoms and order?	What should exist, who authorizes it, and did it safely become real?
Execution	External conformant executor	External executor or Converge	Bounded Loop and settlement
Extra systems	PRE/POST, lifecycle, conformance	Discovery, seams, lanes, legs, graph, Task Pack	Capture, architecture, barrier, Bind, Loop, receipts, recovery, graph, learning

Use Task-Spec when you know the job. Use Seamwise when you know the outcome. Use Converge when you must govern the whole journey.

GLOSSARY

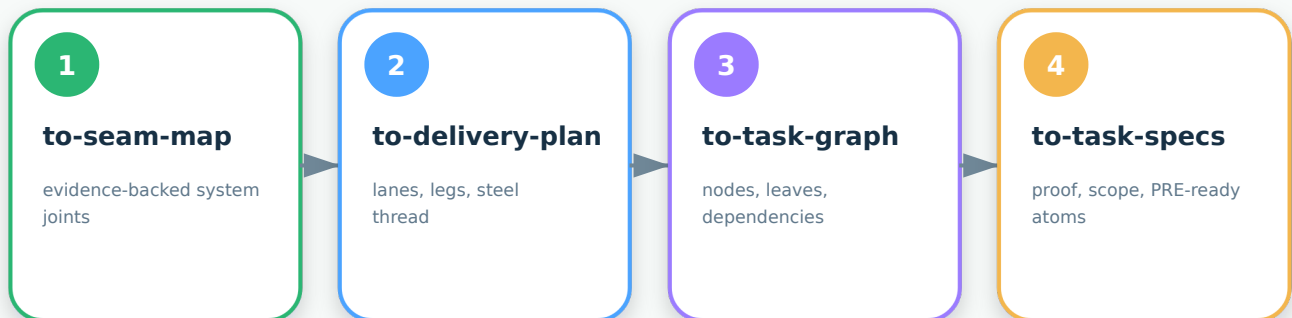
Words that keep the compiler coherent

TERM	CANONICAL MEANING
Delivery Intent	The intended observable outcome, scope, constraints and evidence target.
Seam	A real system joint used to separate coherent responsibility.
Swimlane	The change stream aligned to one accepted seam.
Capability leg	An independently provable capability state inside a swimlane.
Steel thread	The minimum cross-lane path proving end-to-end value early.
Task graph	Composition nodes, executable leaves, prerequisite edges, lineage and contention.
Task-Spec	A vendor-portable atomic work contract with executable proof.
Task Pack	The embedded Seamwise implementation of Task-Spec format, gates and lifecycle.
Decomposition Resolution	The gate that folds false seams, adds integration legs, creates nodes, or stops for decisions.
Convergence Graph	A rebuildable projection connecting canonical artifacts with execution evidence.
Lesson	A scoped, reviewed, versioned, test-backed improvement candidate.

CLOSING

The canonical implementation thesis

Four transformations. One intent-to-task compiler.



The first three decide what the work is. The fourth materializes each trustworthy atom.

Seamwise understands, structures, decomposes, and specifies.

Converge authorizes, executes, proves, settles, and learns.

Prepared from the uploaded Converge canonical handbook release 0.1.0, Task-Spec package 0.1.0 / format contract 3, and the architecture decisions developed in this design conversation. Current, proposed, and future claims are intentionally distinguished.